

Paper title

CLASP: Cost-Optimized LLM-based Agentic System for Phishing Detection

Authors & affiliations

Fouad Trad
Electrical and Computer Engineering
American University of Beirut
Beirut, Lebanon
fat10@mail.aub.edu

Ali Chehab
Electrical and Computer Engineering
American University of Beirut
Beirut, Lebanon
chehab@aub.edu.lb

Intro and motivation

Introduce approach and summarize method

Summarize key results and findings

keywords

Introduce the topic. You can add citations

Highlight the key challenges with the current approaches that solve your problem.

Abstract—Phishing websites remain a significant cybersecurity threat, necessitating accurate and cost-effective detection mechanisms. In this paper, we present CLASP, a novel system that effectively identifies phishing websites by leveraging multiple intelligent agents, built using large language models (LLMs), to analyze different aspects of a web resource. The system processes URLs or QR codes, employing specialized LLM-based agents that evaluate the URL structure, webpage screenshot, and HTML content to predict potential phishing threats. To optimize performance while minimizing operational costs, we experimented with multiple combination strategies for agent-based analysis, ultimately designing a strategic combination that ensures the per-website evaluation expense remains minimal without compromising detection accuracy. We tested various LLMs, including Gemini 1.5 Flash and GPT-4o mini, to build these agents and found that Gemini 1.5 Flash achieved the best performance with an F1 score of 83.01% on a newly curated dataset. Also, the system maintained an average processing time of 2.78 seconds per website and an API cost of around \$3.18 per 1,000 websites. Moreover, CLASP surpasses leading previous solutions, achieving over 40% higher recall and a 20% improvement in F1 score for phishing detection on the collected dataset. To support further research, we have made our dataset publicly available, supporting the development of more advanced phishing detection systems.

Index Terms—Phishing Detection, Multi-Agent System, Large Language Models (LLMs), Cost-Optimization

I. INTRODUCTION

Phishing attacks remain one of the most prevalent and damaging cybersecurity threats, targeting individuals, organizations, and critical infrastructure alike. These attacks often take the form of fraudulent websites that impersonate legitimate services to deceive users into revealing sensitive information such as passwords, financial details, and personal data. As phishing tactics evolve, the methods employed to detect and mitigate such threats must also advance. Traditional phishing detection techniques often rely on a combination of heuristic analysis and URL inspection, but the growing sophistication of phishing websites demands more advanced, resource-efficient solutions that can adapt to emerging attack strategies [1]. Machine learning-based solutions have proven effective in addressing some of these challenges by identifying complex patterns and adapting to new threats. However, they require continuous training and maintenance to remain effective against evolving phishing tactics [2]. Prompt-engineered large

language models (LLMs) offer an alternative by leveraging their extensive pretraining and adaptability through carefully designed prompts, eliminating the need for model retraining [3]. While this approach enhances phishing detection flexibility and reduces maintenance overhead, it introduces the challenge of API costs, as LLM-based systems rely on external providers for processing [4]. Additionally, since these models are not fine-tuned specifically for phishing detection, they may not perform as well as dedicated machine learning models [5], [6]. The challenge, therefore, lies in designing effective prompting strategies that maximize their performance and reliability in detecting phishing websites [7].

In this paper, we propose CLASP, a novel system that utilizes multiple intelligent agents, built on LLMs, to detect phishing websites. Our system processes various aspects of a website, including the URL, webpage screenshot, and HTML content, and employs specialized agents to analyze each of these components individually. The goal is to provide accurate and cost-effective phishing detection by leveraging the unique strengths of LLMs to evaluate different facets of a web resource. Since LLMs are used through prompting rather than training or fine-tuning, and their providers continuously update them, our system eliminates the need for model retraining while still adapting to evolving phishing tactics. By combining the results of these agents, we aim to achieve improved performance without significantly increasing API costs.

To establish a baseline, we first tested individual agents that analyzed only one component at a time: the URL, the webpage screenshot, or the HTML content. Each of these methods, while useful on its own, exhibited limited performance in terms of detection accuracy. This was expected, as phishing websites often exhibit a combination of cues across different components that, when analyzed in isolation, may not provide sufficient information to identify them accurately.

Next, we explored different methods for combining the results of the individual agents. We initially experimented with majority voting, where the final prediction was based on the most common result across all agents. However, this approach did not yield the desired performance. We then tested a sequential processing strategy, where agents analyze the website progressively, starting with the URL, followed by the webpage screenshot, and finally the HTML content. If any

Summarize the approach and how it addresses the previously mentioned challenges

agent flags the website as phishing at any stage, the system immediately classifies it as phishing, eliminating unnecessary calls to subsequent agents. Our findings revealed that the progressive combination strategy not only provided the best detection performance but also maintained a low operational cost. On average, this method processed a website in just 2.78 seconds, with a cost of \$3.18 per 1,000 websites. This combination strategy represents an effective trade-off between accuracy and resource efficiency, making it a promising solution for large-scale phishing detection applications.

Mention the key results

Evaluated on a newly curated dataset, CLASP outperforms existing commercial phishing detection solutions by over 40% in recall and 20% in F1 score, demonstrating the potential of leveraging multiple LLM-based agents for enhanced phishing detection. To foster further research and development in this area, we have made our dataset publicly available, enabling other researchers to build upon and improve our approach.

In summary, the main contributions of this work are:

- 1) Development of CLASP, a multi-agent phishing detection system that analyzes multiple components of a phishing website, including the URL, webpage screenshot, and HTML content, to improve detection performance.
- 2) Evaluation of different agent-based techniques for combining agent outputs.
- 3) Balancing detection accuracy with resource efficiency, achieving an average processing time of 2.78 seconds per website and a cost of \$3.18 per 1,000 websites, making our approach highly cost-effective for large-scale deployment.
- 4) Outperforming leading commercial phishing detection products by more than 20%, showcasing the potential of leveraging multi-agent LLM-based approaches in real-world applications.
- 5) Curating a new phishing dataset and making it publicly available to support further research and development in this area.

Contributions should be explicitly listed as bullet points

The remainder of the paper is organized as follows: Section 2 provides the background and preliminaries essential to our study. Section 3 reviews related work. Section 4 outlines the methodology, while Section 5 details the experimental setup and presents the results, discussing the findings from both performance and cost perspectives. Finally, Section 6 concludes the paper and suggests directions for future research.

Short outline for the remaining sections

II. BACKGROUND AND PRELIMINARIES

A. Large Language Models (LLMs)

LLMs are advanced artificial intelligence systems designed to interpret and generate text that closely resembles human language. These models are primarily built on transformer architectures, which enable them to process extensive textual data and uncover intricate language patterns [8]. Well-known examples of LLMs include OpenAI's GPT models [9] and Google's BERT [10]. LLMs have demonstrated exceptional performance in various natural language processing (NLP)

Background section is not always required, but good to have a brief one

tasks, such as text generation, language translation, and answering questions, due to their ability to utilize vast datasets and sophisticated training methodologies [11].

B. Agentic AI Systems

Agentic AI systems involve the collaboration of multiple autonomous agents working together towards a common objective. In the world of LLMs, these multi-agent systems can enhance the overall performance of the models by allowing specialized agents to focus on different aspects of input data [12]. For instance, one agent may process linguistic features, another might analyze visual elements, and yet another could manage contextual understanding. By distributing tasks among multiple agents and enabling them to communicate and coordinate, these systems can deliver more accurate and efficient results than relying on a single, monolithic model [13]. However, creating such systems presents challenges, including ensuring seamless communication, coordination, and trust among agents. Effective collaboration is essential for ensuring that the agents align their efforts toward the system's overarching objectives and make decisions that benefit the overall application.

III. RELATED WORK

Phishing detection has evolved significantly over the years, with early methods relying heavily on traditional heuristic and rule-based approaches [1], [14]. These systems often focused on identifying known malicious URLs or applying keyword-based filters to detect phishing websites [15], [16]. While these foundational methods provided a starting point, they were often prone to high false-positive rates and struggled against more advanced phishing attempts that closely mimic legitimate websites or communications [15].

As machine learning began to play a larger role in phishing detection, more advanced models such as decision trees [17], support vector machines [18], and ensemble methods [19] emerged. These models leveraged large datasets to identify patterns in phishing URLs, which improved their detection accuracy. Deep learning techniques, including convolutional neural networks (CNNs) and recurrent neural networks (RNNs), were also applied to analyze more complex data, such as images and natural language [20], [21]. These deep learning models generally outperformed traditional methods, achieving higher accuracy in phishing detection. However, they required substantial computational resources and continuous retraining, making them less feasible for real-time or large-scale deployment.

Summarize previous work by citing the relevant papers

In recent years, the use of LLMs in cybersecurity applications has gained significant attention [22], [23]. Prompt-engineered LLMs, in particular, have demonstrated considerable potential for phishing detection due to their ability to adapt dynamically using specific input prompts, without the need for ongoing retraining [3], [6]. These models have been applied to analyze textual data from phishing URLs [24], offering improvements over traditional machine learning approaches.

Highlight the gap we try to address

While the application of LLMs to phishing detection is promising, previous studies have primarily focused on analyzing a single modality, such as the URL alone, or the HTML content of websites. However, the integration of LLMs with multiple agents to process different aspects of a website for phishing detection remains largely unexplored. Furthermore, while research on LLMs in phishing detection has gained traction, there is limited work on evaluating the cost of these models, particularly regarding the computational and financial costs associated with API-based systems [4], [25].

Say how we address the gap (what differs between our work and others' works).

This work introduces a novel multi-agent LLM-based system that analyzes multiple aspects of phishing websites, including URLs, HTML content, and screenshots, to improve detection accuracy. Unlike traditional approaches that rely on a single type of data, our system combines the strengths of multiple agents, each specialized in a different website component, to make more informed predictions. Moreover, the system is designed to optimize both detection performance and cost-efficiency, addressing the economic challenges associated with API usage by minimizing the number of queries required for each website.

IV. METHODOLOGY

Detailed overview of the proposed system and methods with relevant diagrams that show system components and how they are connected

The proposed system leverages three LLM-based agents that analyze distinct data representations (URL, Screenshot, and HTML) of a given website to predict phishing attempts. The Screenshot Agent inspects the visual presentation to identify deceptive branding or misleading elements commonly seen in phishing websites. The URL Agent analyzes the URL's structure and characteristics to assess its potential as a phishing link. The HTML Agent evaluates the webpage's content to detect suspicious patterns, malicious scripts, or phishing indicators. The process begins with the system accepting either a URL or a QR code as input. If a QR code is provided, it is first decoded to extract the corresponding URL.

Initially, we tested each agent on its own to evaluate their baseline performances. Then, we explored multiple strategies to combine the agents' outputs. We started with a Majority Voting Strategy, where a phishing label was assigned if at least two out of the three agents predicted phishing. The process is summarized in Figure 1 (a). This method aims to improve detection robustness by ensuring that phishing identification relies on consensus among the agents. This strategy incurs higher operational costs due to the need to always extract and analyze all three data types: URL, screenshot, and HTML content.

To improve cost efficiency, we adopted a Progressive Analysis Strategy. This method starts by analyzing the URL alone through the URL Agent. If the URL agent does not detect phishing, the system proceeds to extract a full-page screenshot for analysis by the Screenshot Agent. If this agent also fails to identify phishing characteristics, only then the full HTML content is extracted and processed by the HTML Agent. The process is summarized in Figure 1 (b). This progressive approach minimizes resource consumption by limiting data extraction steps and agent analyses to what is necessary.

To assess these methods, we measured detection performance using metrics such as accuracy, F1 score, precision, and recall. Additionally, we evaluated cost efficiency by calculating the average API expenses and monitored the time taken to analyze each website to ensure practical usability.

Part A is essential for reproducibility

V. EXPERIMENTS

Parts B, C, D, E, F, and G could be put as such (results within experiments), or could be separated into 2 sections: Experiments and Results.

A. Experimental Setup

1) **Data:** To evaluate our system, we utilized a subset of the PhiUSIIL dataset, which comprises recently collected websites labeled as legitimate or phishing [26]. For each website, we extracted its screenshot and HTML content; these extracted data elements will be made publicly available to support future research. The dataset can be accessed here. Due to the nature of phishing websites, some URLs were inaccessible at the time of data collection (either because the phishing site had been taken down or due to server unavailability) resulting in a final sample of 2,126 websites (1,110 legitimate and 1,016 phishing).

2) **Models:** For models, we began by building our agents using Gemini 1.5 Flash. After identifying the best-performing setup, we evaluated it on GPT-4o mini to gain insights into how different models perform in this context. These models were selected for their accessibility via API and their relatively lower costs compared to other models, making them particularly viable for industry applications that aim to mitigate phishing threats while maintaining manageable operational costs.

3) **Prompts:** We employed structured output prompts designed to instruct the models to act as "cybersecurity professionals" investigating different website components. Each prompt directed the model to classify the content as either "Phishing" or "Legitimate" while providing a succinct reasoning for the decision. This reasoning is essential for helping users understand why a given website is flagged as phishing or deemed safe. For example, the prompt used for the URL agent was:

"Act as a professional cybersecurity analyst reviewing URLs. Classify the following URL as Phishing or Legitimate. Your reasoning should be succinct"

Similar prompts were adapted for the screenshot and HTML agents, ensuring each agent's task was clearly defined to maximize performance.

B. Assessing Baseline Agents

In this initial experiment, we evaluated the performance of individual agents that analyze distinct website components: URL, screenshot, and HTML content.

The results, shown in Table I, demonstrate that the URL Agent achieved the strongest overall performance among the individual agents, with an accuracy of 0.799 and an F1 score of 0.773. This agent's relatively balanced precision (0.840) and recall (0.717) enabled it to detect a reasonable portion of phishing websites without excessive false positives.

The Screenshot Agent and HTML Agent achieved even higher precision values (0.884 and 0.967, respectively), but

Experiments could be merged with the methods BUT don't put results in methods

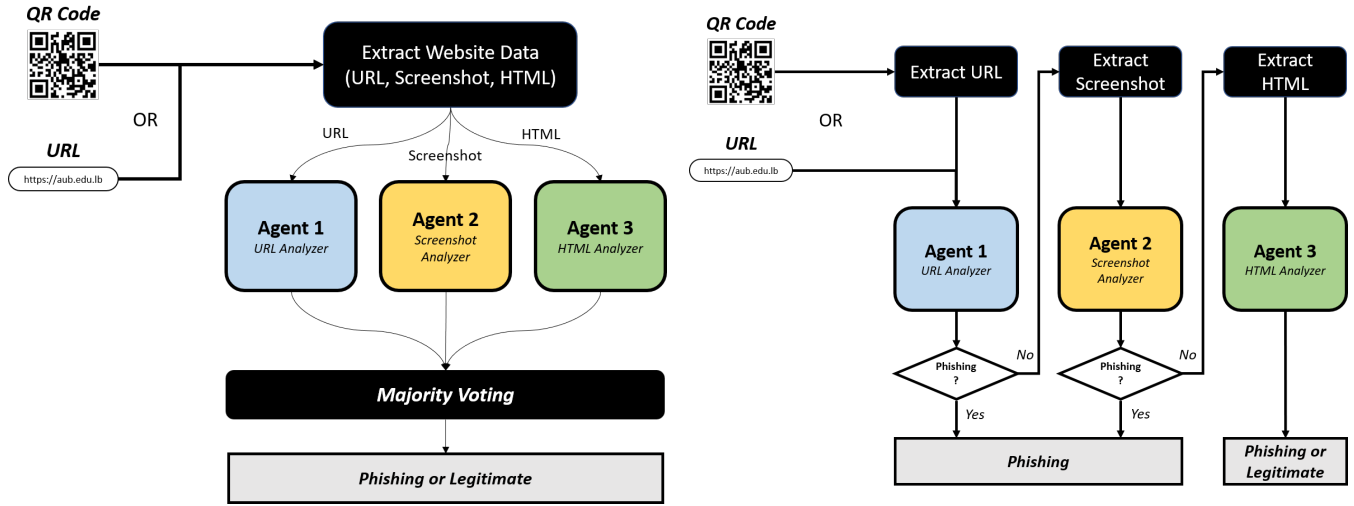


Fig. 1. Comparison of (a) Majority Voting and (b) Progressive Analysis.

their recall rates were notably lower (0.457 and 0.483). This reveals that while these agents excel at confirming phishing attempts when identified, they are less effective at detecting harder-to-identify phishing websites, reducing their overall reliability as standalone solutions.

These results highlight a common challenge in phishing detection: while achieving high precision is important for avoiding false positives, poor recall significantly limits the system’s ability to capture a wider range of phishing threats.

Table II presents the resource consumption of different agents, with times referring only to the analysis phase and not including the time required for capturing screenshots or saving HTML content. In terms of resource efficiency, the URL Agent proved to be the fastest and most cost-effective, processing a URL in 0.68 seconds at a cost of \$0.018 per 1,000 websites. The HTML Agent, while achieving higher precision, required significantly more resources, averaging 2.28 seconds per website at \$4.697 per 1,000 websites. The Screenshot Agent had an intermediate processing time of 1.18 seconds. These findings highlight the need for strategies that optimize both detection performance and resource efficiency.

C. Majority Voting

In this experiment, we explored a Majority Voting strategy, where a website is classified as phishing if at least two out of the three agents predict it as such.

However, as shown in Table I, Majority Voting did not result in improved performance compared to the best-performing individual agent (the URL Agent). The Majority Voting approach achieved an accuracy of 0.782, an F1 score of 0.712, and a recall of only 0.563, which is lower than the URL Agent’s recall (0.717).

This outcome reflects the challenge of combining models with differing strengths. While the HTML and Screenshot agents demonstrated strong precision, their lower recall limited their ability to identify phishing websites effectively. Conse-

quently, combining their outputs with the URL Agent caused the ensemble to miss more phishing websites than desired.

In addition to these performance limitations, Majority Voting was notably inefficient. As shown in Table II, it required 4.14 seconds per website and incurred a cost of \$4.753 per 1,000 websites, making it the most resource-intensive method evaluated.

These findings demonstrate that while Majority Voting is often beneficial in ensemble models, its effectiveness can diminish when the combined agents exhibit highly imbalanced strengths, as was the case here. This motivated the development of a more efficient and targeted combination strategy.

D. Progressive Analysis

To enhance detection performance while minimizing resource consumption, we introduced the Progressive Analysis strategy, as highlighted in the Methodology section. This approach follows a staged evaluation process, prioritizing lower-cost analyses first to avoid unnecessary expenses. Unlike Majority Voting, which processes all data sources simultaneously, Progressive Analysis selectively escalates evaluation only when needed, reducing redundant data extraction and analysis.

This staged approach significantly improved both performance and efficiency. As shown in Table I, Progressive Analysis achieved the highest performance across all metrics, with an accuracy of 0.834, precision of 0.814, and a notably improved recall of 0.847. The method’s F1 score of 0.830 reflects the best balance between precision and recall, demonstrating its robustness in detecting phishing threats.

In addition to its strong detection capabilities, Progressive Analysis was also more efficient than Majority Voting. Table II shows that this method required only 2.78 seconds per website, with a reduced cost of \$3.184 per 1,000 websites. By selectively activating agents based on previous outcomes, Progressive Analysis avoided redundant processing, achieving the best trade-off between performance and cost.

These results demonstrate that Progressive Analysis is an effective solution for improving phishing detection rates while optimizing resource efficiency, making it well-suited for real-world deployment.

TABLE I
PERFORMANCE METRICS FOR BASELINES AND AGENT COMBINATIONS

	Accuracy	Precision	Recall	F1 score
URL Agent	0.799	0.840	0.717	0.773
Screenshot Agent	0.712	0.884	0.457	0.602
HTML Agent	0.745	0.967	0.483	0.644
Majority Voting	0.782	0.966	0.563	0.712
Progressive Analysis	0.834	0.814	0.847	0.830

TABLE II
RESOURCE METRICS FOR DIFFERENT AGENTS

	Avg. Time per website (s)	Avg. Cost per website (\$)	Price for 1k websites (\$)
URL Agent	0.68	0.00001823	0.018
Screenshot Agent	1.18	0.00003769	0.038
HTML Agent	2.28	0.00469667	4.697
Majority Voting	4.14	0.00475260	4.753
Progressive Analysis	2.78	0.00318400	3.184

E. Testing with GPT-4o mini

We repeated the Progressive Analysis experiment using GPT-4o mini to gain insights into its performance. The results were very similar to those obtained with the previous model, achieving an accuracy of 0.831, a precision of 0.807, a recall of 0.850, and an F1 score of 0.828. These findings suggest that GPT-4o mini maintains a comparable balance between precision and recall, making it a viable alternative without significant trade-offs in detection quality. For deployment, however, we opted for Gemini 1.5 Flash. This model was chosen because it offers a similar performance to GPT-4o mini but at a lower cost, aligning with our goal of maintaining efficiency while reducing expenses. By selecting this model, we can optimize resource usage without sacrificing the quality of detection. While this model works well for our current needs, we can also explore more sophisticated models in the future that might offer improved performance, though they may come at a higher cost.

F. Comparison with commercial tools

The results of the comparison between our approach and other commercial phishing detection tools available through VirusTotal are summarized in Table III. Some tools did not classify certain websites, leading to their exclusion from metric computations. However, these cases were minimal and did not affect the overall comparisons. While many tools, like Lionic and alphaMountain.ai, achieve near-perfect precision, they exhibit significantly lower recall values. For instance, Lionic has a recall of only 0.342, and alphaMountain.ai achieves a recall of 0.238. This indicates that while these tools are highly precise, they are less effective at identifying phishing websites, leading to a higher number of false negatives.

In contrast, CLASP demonstrates a strong balance between precision and recall. With an accuracy of 0.834 and a recall of 0.847, our approach excels in detecting phishing websites, even at the expense of a slightly lower precision (0.814). This trade-off highlights our model’s ability to identify phishing attempts that other tools might miss, making it more reliable for real-world applications where false negatives can be particularly harmful.

The F1 score, which accounts for both precision and recall, further underscores the advantage of our approach. With an F1 score of 0.830, our system outperforms most commercial solutions, including BitDefender (0.608) and G-Data (0.597), indicating that it provides a more balanced and effective phishing detection solution.

In conclusion, while other tools may excel in precision, our system achieves a better balance between precision and recall, outperforming commercial solutions by 40% in recall and 20% in F1 score. This leads to higher overall performance and significantly fewer missed phishing websites, making our approach a more robust and reliable solution for phishing detection.

TABLE III
COMPARISON OF PHISHING DETECTION PERFORMANCE OF DIFFERENT TOOLS AND OUR APPROACH.

Tool	Accuracy	Precision	Recall	F1 score
Lionic	0.688	1.000	0.342	0.510
alphaMountain.ai	0.646	1.000	0.238	0.384
Antiy-AVL	0.594	1.000	0.144	0.252
BitDefender	0.733	1.000	0.437	0.608
CyRadar	0.679	1.000	0.311	0.474
Dr.Web	0.579	1.000	0.114	0.205
Emsisoft	0.561	1.000	0.076	0.141
ESET	0.643	1.000	0.247	0.396
Fortinet	0.718	1.000	0.407	0.578
G-Data	0.727	1.000	0.426	0.597
Kaspersky	0.641	1.000	0.243	0.391
Phishtank	0.594	1.000	0.144	0.252
Seclookup	0.768	1.000	0.432	0.604
Sophos	0.720	1.000	0.411	0.582
Trustwave	0.560	1.000	0.072	0.135
Webroot	0.707	0.989	0.375	0.544
Forcepoint ThreatSeeker	0.690	0.989	0.339	0.504
CLASP (Our System)	0.834	0.814	0.847	0.830

G. Deployment Here is where we include a link for the demo

Based on the high performance of CLASP, we deployed it in a web application that allows users to test URLs or QR codes and determine their phishing status. The system not only returns a classification result but also enhances explainability by presenting reasoning generated by each LLM agent. These explanations highlight specific features that influenced the decision, such as suspicious URL patterns, structural anomalies in the webpage, or phishing-related visual elements. A screenshot of a detected phishing website is shown in Figure 2, while Figure 3 presents a legitimate website obtained from a scanned QR code. Additionally, a demo of the application can be accessed at <https://youtu.be/s9CcJ6jiYZU>, and it will be deployed for public use after further optimizations to enhance its efficiency and reliability.

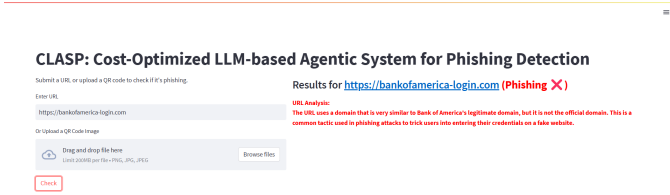


Fig. 2. Screenshot of a detected phishing website.

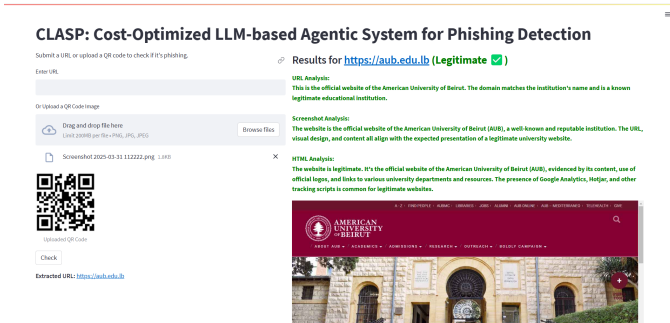


Fig. 3. Screenshot of a legitimate website obtained from a scanned QR code.

VI. CONCLUSION

In this study, we introduced CLASP, a phishing detection system that balances accuracy and resource efficiency through Progressive Analysis. Our evaluation against commercial phishing detection tools demonstrated that CLASP outperforms them by more than 20% in F1 score and 40% in recall while maintaining high precision, addressing a critical limitation of existing solutions that often fail to detect a significant number of phishing websites. The system has been successfully deployed in a web application, enabling users to analyze URLs and QR codes efficiently.

Future work will focus on further improving detection accuracy by integrating more advanced language models and refining the Progressive Analysis strategy to reduce false negatives. These improvements will ensure CLASP remains a robust and scalable solution for combating evolving cybersecurity threats. Additionally, we plan to deploy the application for public use, making it accessible to a wider audience.

ACKNOWLEDGMENT

The authors would like to acknowledge that this work has been supported by the Maroun Semaan Faculty of Engineering at the American University of Beirut.

REFERENCES

[1] C. M. R. da Silva, E. L. Feitosa, and V. C. Garcia, "Heuristic-based strategy for phishing prediction: A survey of url-based approach," *Computers & Security*, vol. 88, p. 101613, 2020.

[2] L. Tang and Q. H. Mahmoud, "A survey of machine learning-based solutions for phishing website detection," *Machine Learning and Knowledge Extraction*, vol. 3, no. 3, pp. 672–694, 2021.

[3] F. Trad and A. Chehab, "To ensemble or not: Assessing majority voting strategies for phishing detection with large language models," in *International Conference on Intelligent Systems and Pattern Recognition*, pp. 158–173, Springer, 2024.

[4] F. Trad and A. Chehab, "Large multimodal agents for accurate phishing detection with enhanced token optimization and cost reduction," in *2024 2nd International Conference on Foundation and Large Language Models (FLLM)*, pp. 229–237, IEEE, 2024.

[5] F. Trad and A. Chehab, "Evaluating the efficacy of prompt-engineered large multimodal models versus fine-tuned vision transformers in image-based security applications," *ACM Transactions on Intelligent Systems and Technology*, May 2025.

[6] F. Trad and A. Chehab, "Prompt engineering or fine-tuning? a case study on phishing detection with large language models," *Machine Learning and Knowledge Extraction*, vol. 6, no. 1, pp. 367–384, 2024.

[7] F. Trad and A. Chehab, "Manual prompt engineering is not dead: A case study on large language models for code vulnerability detection with dspy," in *2025 8th International Conference on Data Science and Machine Learning Applications (CDMA)*, pp. 168–173, IEEE, 2025.

[8] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong, et al., "A survey of large language models," *arXiv preprint arXiv:2303.18223*, 2023.

[9] K. I. Roumeliotis and N. D. Tselikas, "Chatgpt and open-ai models: A preliminary review," *Future Internet*, vol. 15, no. 6, p. 192, 2023.

[10] J. Devlin, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.

[11] Y. Chang, X. Wang, J. Wang, Y. Wu, L. Yang, K. Zhu, H. Chen, X. Yi, C. Wang, Y. Wang, et al., "A survey on evaluation of large language models," *ACM Transactions on Intelligent Systems and Technology*, vol. 15, no. 3, pp. 1–45, 2024.

[12] J. Xie, Z. Chen, R. Zhang, X. Wan, and G. Li, "Large multimodal agents: A survey," *arXiv preprint arXiv:2402.15116*, 2024.

[13] K. Sreedhar and L. Chilton, "Simulating human strategic behavior: Comparing single and multi-agent llms," *arXiv preprint arXiv:2402.08189*, 2024.

[14] I. Qabajeh, F. Thabtah, and F. Chiclana, "A recent review of conventional vs. automated cybersecurity anti-phishing techniques," *Computer Science Review*, vol. 29, pp. 44–55, 2018.

[15] Y. Ding, N. Luktarhan, K. Li, and W. Slamu, "A keyword-based combination approach for detecting phishing webpages," *computers & security*, vol. 84, pp. 256–275, 2019.

[16] C. L. Tan, K. L. Chiew, K. Wong, et al., "Phishwho: Phishing webpage detection via identity keywords extraction and target domain name finder," *Decision Support Systems*, vol. 88, pp. 18–27, 2016.

[17] L. Machado and J. Gadge, "Phishing sites detection based on c4. 5 decision tree algorithm," in *2017 International Conference on Computing, Communication, Control and Automation (ICCUBEA)*, pp. 1–5, IEEE, 2017.

[18] A. Altaher, "Phishing websites classification using hybrid svm and knn approach," *International Journal of Advanced Computer Science and Applications*, vol. 8, no. 6, 2017.

[19] A. A. Ubung, S. K. B. Jasmi, A. Abdullah, N. Jhanjhi, and M. Supramaniam, "Phishing website detection: An improved accuracy through feature selection and ensemble learning," *International Journal of Advanced Computer Science and Applications*, vol. 10, no. 1, 2019.

[20] Z. Alshingiti, R. Alaql, J. Al-Muhtadi, Q. E. U. Haq, K. Saleem, and M. H. Faheem, "A deep learning-based phishing detection system using cnn, lstm, and lstm-cnn," *Electronics*, vol. 12, no. 1, p. 232, 2023.

[21] X. Xiao, D. Zhang, G. Hu, Y. Jiang, and S. Xia, "Cnn-mhsa: A convolutional neural network and multi-head self-attention combined approach for detecting phishing websites," *Neural Networks*, vol. 125, pp. 303–312, 2020.

[22] J. Zhang, H. Bu, H. Wen, Y. Chen, L. Li, and H. Zhu, "When llms meet cybersecurity: A systematic literature review," *arXiv preprint arXiv:2405.03644*, 2024.

[23] F. N. Motlagh, M. Hajizadeh, M. Majd, P. Najafi, F. Cheng, and C. Meinel, "Large language models in cybersecurity: State-of-the-art," *arXiv preprint arXiv:2402.00891*, 2024.

[24] M. A. Ferrag, F. Alwahedi, A. Battah, B. Cherif, A. Mechri, and N. Tihanyi, "Generative ai and large language models for cyber security: All insights you need," *arXiv preprint arXiv:2405.12750*, 2024.

[25] J. Wang, S. Jain, D. Zhang, B. Ray, V. Kumar, and B. Athiwaratkun, "Reasoning in token economies: Budget-aware evaluation of llm reasoning strategies," *arXiv preprint arXiv:2406.06461*, 2024.

[26] A. Prasad and S. Chandra, "Phiusiil: A diverse security profile empowered phishing url detection framework based on similarity index and incremental learning," *Computers & Security*, vol. 136, p. 103545, 2024.

Summarize the work and its main findings again

Mention how it could be extended in future works

I advise you to use a reference manager for the references (Zotero, Mendeley, etc.)